Protocols Implementation

HTTP and Gopher Protocols from Scratch in C

A Line-by-Line Technical Walkthrough

Mohammed Bakr

July 20, 2026

Repository: https://github.com/MohammedBakr1/Protocols_Implementation
Language: C (C11 standard)
Platform: Linux (POSIX sockets API)
License: MIT

Contents

Chapter 1

Introduction

1.1 Purpose of This Project

This project implements two application-layer network protocols entirely from scratch in C, using nothing but the raw POSIX sockets API:

- **Gopher** (RFC 1436) — a minimalist document-retrieval protocol from 1991.
- **HTTP/1.1** (a practical subset) — the protocol underlying the modern web.

For each protocol, both a **server** and a **client** are implemented. No external networking libraries are used; every byte sent or received passes through hand-written code that calls directly into the operating system's socket layer (`socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`).

1.2 Why These Two Protocols Together

Gopher and HTTP were chosen deliberately as a pair, not arbitrarily:

- **Gopher is almost the simplest possible TCP application protocol.** A request is a single line of text; a response is raw bytes followed by connection closure. There are no headers, no status codes, and no explicit content length. This makes it an ideal first target: it forces you to master the TCP connection lifecycle (`accept`/`connect`, `recv` loops, `send` loops, closing sockets) without the added complexity of message framing.
- **HTTP adds exactly the complexity that Gopher lacks.** Once the TCP fundamentals are solid, HTTP introduces the next layer of concepts that essentially every real-world protocol needs: structured request/response lines, key–value headers, explicit content-length framing, and status codes. Implementing HTTP after Gopher means each new concept is introduced in isolation, rather than all at once.

The result is a natural progression: Chapter 5 onward builds the simple case, and Chapter 8 onward builds on top of the same socket primitives to handle the harder case.

1.3 Learning Goals

By the end of working through this project and this document, the reader should be able to:

1. Explain the full lifecycle of a TCP server socket (`socket` → `bind` → `listen` → `accept`) and a TCP client socket (`socket` → `connect`).

2. Explain why `recv()` and `send()` do not guarantee delivering or accepting all requested bytes in a single call, and why loops are required around both.
3. Implement a simple line-based protocol (Gopher) that relies on connection closure to signal the end of a message.
4. Implement a framed protocol (HTTP) that relies on an explicit `Content-Length` header, including correctly separating headers from body in a byte stream.
5. Understand basic server-side input validation (e.g. path traversal protection) and why client-side behavior (e.g. URL normalization in `curl`) cannot be relied upon for security.
6. Build and run all of the above on a Linux system, including troubleshooting the most common pitfalls (buffering, working directory issues, background process management).

1.4 Prerequisites

The reader is assumed to have:

- Working knowledge of C (pointers, structs, manual memory management with `malloc`/`free`).
- Basic familiarity with the command line on Linux.
- No prior networking experience is assumed — Chapter 2 builds the necessary background from first principles.

1.5 Document Structure

- **Chapter 2** builds the conceptual foundation: what a socket is, the TCP three-way handshake, byte ordering, and blocking I/O.
- **Chapter 3** gives a bird's-eye view of the repository layout.
- **Chapter 4** walks through the shared utility library used by every server and client in the project.
- **Chapters 5–7** cover the Gopher protocol, server, and client in full detail.
- **Chapters 8–10** cover HTTP in the same depth.
- **Chapter 11** explains the `Makefile`.
- **Chapter 12** is a practical, copy-pasteable guide to building and running everything on Linux.
- **Chapter 13** documents how each component was verified to work correctly.
- **Chapter 14** is an honest account of what this implementation deliberately does *not* handle, and what a production-grade version would need to add.

Chapter 2

Networking Fundamentals

This chapter builds the conceptual vocabulary used throughout the rest of the document. Readers already comfortable with sockets, TCP, and byte ordering may skip to Chapter 3.

2.1 What Is a Socket?

A **socket** is an operating-system abstraction representing one endpoint of a bidirectional communication channel. From the perspective of a C program, a socket behaves much like a file: the kernel hands back an integer **file descriptor** (fd), and the program reads from and writes to that descriptor using system calls. The key difference is that instead of reading/writing bytes to a file on disk, the bytes travel across a network (or, in the case of `localhost`, through a loopback interface without ever leaving the machine).

Every socket used in this project is created with:

```
1 int fd = socket(AF_INET, SOCK_STREAM, 0);
```

- `AF_INET` selects the IPv4 address family.
- `SOCK_STREAM` selects TCP: a connection-oriented, reliable, ordered byte stream.
- The third argument (`0`) lets the kernel pick the default protocol for the given family/type combination, which for `SOCK_STREAM` is always TCP.

Note

This project exclusively uses TCP (`SOCK_STREAM`). The alternative, `SOCK_DGRAM` (UDP), provides no ordering or delivery guarantees and is not used here. Both HTTP and Gopher are defined as TCP protocols.

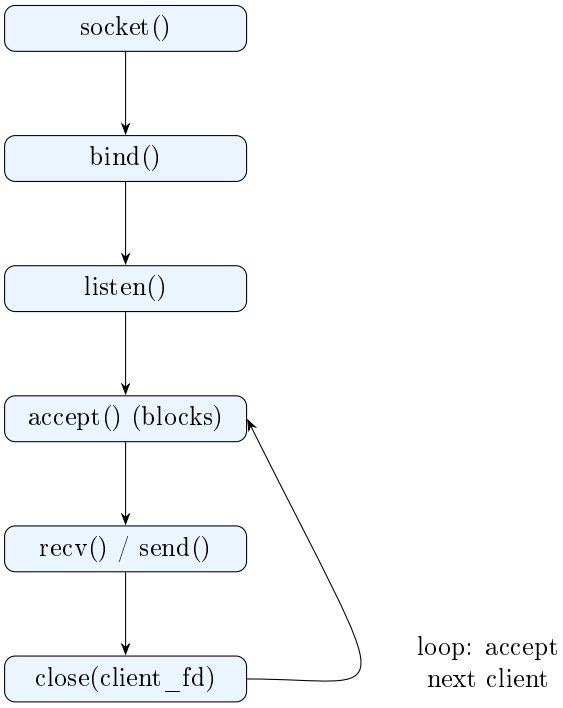
2.2 The TCP Connection Lifecycle

2.2.1 Server Side

A TCP server follows a strict sequence of system calls:

1. `socket()` — create an unbound, unconnected socket.
2. `bind()` — associate the socket with a specific local IP address and port number.

3. `listen()` — mark the socket as passive (willing to accept incoming connections), and specify a backlog queue size.
4. `accept()` — block until a client connects, then return a *new* file descriptor representing that specific connection. The original listening socket remains open and can accept further connections.
5. `recv()` / `send()` — exchange data with the connected client, using the fd returned by `accept()`, not the listening fd.
6. `close()` — close the per-connection fd once the exchange is finished.



2.2.2 Client Side

A client's sequence is much shorter:

1. `socket()` — create the socket.
2. `connect()` — initiate the TCP three-way handshake with a specific server address and port. This call blocks until the handshake completes (or fails).
3. `send()` / `recv()` — exchange data.
4. `close()` — close the connection.

2.3 The TCP Three-Way Handshake

Although the handshake itself is handled transparently by the kernel when `connect()` and `accept()` are called, understanding it explains why these calls can block and occasionally fail:

1. **SYN**: the client sends a segment requesting synchronization.
2. **SYN-ACK**: the server acknowledges and sends its own synchronization request.
3. **ACK**: the client acknowledges the server's request.

Only after this exchange completes does `accept()` on the server return a usable connected file descriptor, and does `connect()` on the client return successfully.

2.4 Byte Order: Why `htons()` Exists

Different CPU architectures store multi-byte integers in memory in different orders:

- **Little-endian** (e.g. x86, x86-64): the least significant byte is stored first.
- **Big-endian** (e.g. many older RISC systems, and network protocols by convention): the most significant byte is stored first.

Network protocols standardize on big-endian, historically called **network byte order**. Because a client and server may run on machines with different native (**host**) byte orders, port numbers and IP addresses must be explicitly converted before being placed into a `struct sockaddr_in`. This project uses:

```
addr.sin_port = htons(port);
```

`htons` stands for **h**ost **to** **n**etwork, **s**hort (16-bit). The corresponding reverse conversion is `ntohs`. For 32-bit values (such as IPv4 addresses), the analogous functions are `htonl` and `ntohl`.

Forgetting `htons()` is a classic bug: on a little-endian machine, binding to port 8080 (`0x1F90`) without conversion would actually bind to port `0x901F` = 36895. The code would compile and run without error, but the server would listen on the wrong port.

2.5 Blocking I/O and Partial Reads/Writes

This is the single most important practical lesson in this project, and the reason the shared utility library (Chapter 4) exists at all.

2.5.1 `recv()` May Return Fewer Bytes Than Requested

When a program calls `recv(fd, buffer, 4096, 0)`, it is asking the kernel for *up to* 4096 bytes. The kernel is free to return anywhere from 1 byte to 4096 bytes in a single call, depending on how much data has arrived and been buffered so far. TCP is a *byte stream*, not a message-based protocol: it makes no promise that the boundaries of your `send()` calls on one side will match the boundaries of your `recv()` calls on the other side.

Consequently, any code that assumes "one `recv()` call = one complete message" is incorrect in general, even though it may appear to work reliably in local testing (where messages are small and the loopback interface rarely fragments them).

2.5.2 `send()` May Send Fewer Bytes Than Requested

Symmetrically, `send(fd, buffer, len, 0)` is permitted to transmit fewer than `len` bytes and return the actual number sent. This typically happens when the kernel's internal send buffer is full. Code that ignores the return value of `send()` silently truncates its own output under load.

2.5.3 The Standard Fix: Loop Until Done

The correct pattern for both directions is a loop:

```c
size_t sent_total = 0;
while (sent_total < len) {
    ssize_t n = send(fd, buf + sent_total, len - sent_total, 0);
    if (n < 0) { /* handle error */ }
    sent_total += (size_t)n;
}
```

This project implements exactly this pattern once, in `send_all()` (Chapter 4), and reuses it everywhere data needs to be sent reliably.

2.6 Signaling the End of a Message

Because TCP is a raw byte stream with no built-in concept of "message boundaries," every application-layer protocol must define its own way of telling the receiver when a logical message is complete. This project's two protocols use the two most common strategies, which is precisely why they were chosen together:

- **Connection closure** (used by Gopher): the sender simply closes the connection once done. The receiver knows the message is complete when `recv()` returns `0` (end-of-file on the socket).
- **Explicit length framing** (used by HTTP): the sender includes a `Content-Length` header stating exactly how many body bytes follow the headers. The receiver first finds the end of the headers (the sequence `"\r\n\r\n"`), then reads exactly that many additional bytes.

Both strategies are explored in depth in their respective protocol chapters.

Chapter 3

Project Structure

3.1 Repository Layout

```
Protocols_Implementation/
|-- common/
|   |-- socket_utils.h      Shared socket helper declarations
|   `-- socket_utils.c      Shared socket helper implementation
|-- gopher/
|   |-- gopher_server.c     Gopher server
|   |-- gopher_client.c     Gopher client
|   `-- gopher_root/        Sample files served by the Gopher server
|       |-- README
|       `-- about.txt
|-- http/
|   |-- http_server.c       HTTP server
|   |-- http_client.c       HTTP client
|   `-- www/                Sample files served by the HTTP server
|       |-- index.html
|       `-- about.txt
|-- docs/                   This LaTeX document and its compiled PDF
|   |-- main.tex
|   |-- chapters/
|   `-- Protocols_Implementation.pdf
|-- Makefile
|-- README.md
`-- LICENSE
```

3.2 Design Rationale

3.2.1 Why a Shared `common/` Module?

Both protocols' servers and clients need to perform the same low-level operations: create a listening socket, connect as a client, reliably send a buffer, and (for Gopher) read until the peer closes the connection. Rather than duplicating this logic four times, it is factored into `common/socket_utils.c`, and every other file includes its header and links against its object file. This is standard C practice: a small, focused, reusable module with a clear interface (declared in the `.h` file) and a hidden implementation (in the `.c` file).

3.2.2 Why Separate Directories per Protocol?

Keeping `gopher/` and `http/` as separate directories, each with its own sample-content folder (`gopher_root/` and `www/`), mirrors how real servers are typically organized: application code

separate from the content it serves. It also keeps the two protocol implementations visually and organizationally independent, which matters for a project whose explicit purpose is pedagogical comparison between the two.

3.2.3 Four Independent `main()` Functions

This project intentionally builds four separate executables rather than one combined program with subcommands:

Executable	Source file	Role
`gopher_server`	`gopher/gopher_server.c`	Listens on TCP port 7070, serves Gopher selectors
`gopher_client`	`gopher/gopher_client.c`	Connects to a Gopher server and prints the response
`http_server`	`http/http_server.c`	Listens on TCP port 8080, serves files over HTTP/1.1
`http_client`	`http/http_client.c`	Connects to an HTTP server, sends a GET request, parses the response

Each has its own `int main(int argc, char *argv[])`. This is why, when importing the project into an IDE such as Code::Blocks, it must be imported as a *Makefile project* rather than a single "C project" — a normal single-target C project expects exactly one `main()` across all its source files, and would fail to link with four.

Chapter 4

The Shared Socket Utility Library

This chapter examines `common/socket_utils.h` and `common/socket_utils.c` in full detail. Every other C file in the project depends on this module, so a thorough understanding of it is a prerequisite for the rest of the document.

4.1 The Header File: `socket_utils.h`

```c
#ifndef SOCKET_UTILS_H
#define SOCKET_UTILS_H

int create_server_socket(int port, int backlog);
int connect_to_server(const char *host, int port);
char *read_until_close(int fd, size_t *out_len);
int send_all(int fd, const char *buf, size_t len);

#endif
```

4.1.1 Include Guards

The `#ifndef` / `#define` / `#endif` triplet is a standard **include guard**. If this header is accidentally included more than once within the same translation unit (which can happen indirectly through chains of includes), the guard prevents the compiler from seeing duplicate declarations, which would otherwise be a compile error.

4.1.2 The Four Declared Functions

Function	Purpose
`create_server_socket`	Performs `socket`, `setsockopt`, `bind`, and `listen` in one call; returns a ready-to-`accept` listening file descriptor.
`connect_to_server`	Performs `getaddrinfo`, `socket`, and `connect`; returns a connected file descriptor.
`read_until_close`	Reads from a socket in a loop until the peer closes the connection (EOF), dynamically growing a heap buffer as needed.
`send_all`	Sends an entire buffer over a socket, looping as necessary to handle partial writes (see Section 2.5).

4.2 The Implementation File: `socket_utils.c`

4.2.1 Feature Test Macro and Includes

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include "socket_utils.h"
```

The project is compiled with `-std=c11`, which tells GCC to expose only strictly-standard C library features and hide POSIX-specific extensions such as `getaddrinfo`. Defining `_POSIX_C_SOURCE 200809L` *before any header is included* explicitly requests the POSIX.1-2008 feature set, making functions like `getaddrinfo` and `gai_strerror` visible again. This macro must appear before the very first `#include` line to take effect correctly.

4.2.2 `create_server_socket`

```c
int create_server_socket(int port, int backlog) {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("socket");
        return -1;
    }
```

Creates a TCP/IPv4 socket. `perror` prints a human-readable description of `errno` (the last system-call error code) prefixed with the given string, e.g. `socket: Too many open files`.

```c
    int opt = 1;
    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) <
    0) {
        perror("setsockopt");
        close(server_fd);
        return -1;
    }
```

`SO_REUSEADDR` allows the socket to bind to a port that is still nominally occupied by a recently-closed connection lingering in the `TIME_WAIT` state. Without this option, restarting the server immediately after stopping it often fails with `Address already in use`, because the kernel reserves the port for a short grace period (typically up to a couple of minutes) to catch any stray packets from the previous connection.

```c
    struct sockaddr_in addr;
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(port);

    if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        close(server_fd);
        return -1;
    }
```

`struct sockaddr_in` describes an IPv4 endpoint (address family, port, IP address). It is zero-initialized with `memset` first as defensive practice, since uninitialized padding bytes in the struct

could otherwise contain garbage. `INADDR_ANY` means "bind to all available network interfaces on this machine" (as opposed to a single specific IP). `htons(port)` converts the port number to network byte order, as explained in Section 2.5 of the previous chapter. The cast to `struct sockaddr *` is required because the POSIX sockets API is written in terms of a generic address structure that specific address families (like `sockaddr_in`) are meant to be cast to and from.

```
    if (listen(server_fd, backlog) < 0) {
        perror("listen");
        close(server_fd);
        return -1;
    }

    return server_fd;
}
```

`listen` transitions the socket into passive (listening) mode. `backlog` bounds how many fully-established connections may queue up waiting for the program to call `accept()`; if the queue is full, further connection attempts may be refused or delayed by the kernel.

4.2.3 connect_to_server

```
int connect_to_server(const char *host, int port) {
    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;

    char port_str[16];
    snprintf(port_str, sizeof(port_str), "%d", port);

    int err = getaddrinfo(host, port_str, &hints, &res);
    if (err != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(err));
        return -1;
    }
```

Unlike the server, which always binds to `INADDR_ANY` (a fixed numeric address), the client must resolve a possibly symbolic host name (e.g. `"localhost"` or `"example.com"`) into a concrete address. `getaddrinfo` performs this resolution — including DNS lookups when necessary — and is the modern, protocol-independent replacement for older functions like `gethostbyname`. It takes the port as a *string*, hence the `snprintf` conversion. On success, `res` points to a linked list of candidate addresses; this project simply uses the first one.

```
    int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (fd < 0) {
        perror("socket");
        freeaddrinfo(res);
        return -1;
    }

    if (connect(fd, res->ai_addr, res->ai_addrlen) < 0) {
        perror("connect");
        close(fd);
        freeaddrinfo(res);
        return -1;
    }

    freeaddrinfo(res);
    return fd;
}
```

The socket is created using the family/type/protocol values *returned by* `getaddrinfo` rather than hard-coded constants, which keeps the code correct even if `hints` were later relaxed to allow IPv6. `connect` performs the TCP three-way handshake. `freeaddrinfo` releases the linked list allocated by `getaddrinfo`; note it must be called on *every* exit path, including the error paths, to avoid a memory leak.

4.2.4 `read_until_close`

This function implements the Gopher-style "read until the peer disconnects" pattern discussed in Section 2.5.

```
char *read_until_close(int fd, size_t *out_len) {
    size_t capacity = 4096;
    size_t len = 0;
    char *buf = malloc(capacity);
    if (!buf) return NULL;

    while (1) {
        if (len + 1024 > capacity) {
            capacity *= 2;
            char *new_buf = realloc(buf, capacity);
            if (!new_buf) {
                free(buf);
                return NULL;
            }
            buf = new_buf;
        }
```

The buffer starts at 4096 bytes and doubles in size whenever fewer than 1024 bytes of headroom remain. Doubling (rather than growing by a fixed increment) is the standard amortized-growth strategy used by dynamic arrays: it keeps the total number of `realloc` calls logarithmic in the final size, rather than linear.

If `realloc` fails, the old `buf` pointer is still valid, but the return value `new_buf` is `NULL`. Overwriting `buf` directly with the result of a failed `realloc` would leak the original allocation. This code correctly checks `new_buf` before reassigning `buf`.

```
        ssize_t n = recv(fd, buf + len, capacity - len - 1, 0);
        if (n < 0) {
            perror("recv");
            free(buf);
            return NULL;
        }
        if (n == 0) {
            break;
        }
        len += (size_t)n;
    }

    buf[len] = '\0';
    if (out_len) *out_len = len;
    return buf;
}
```

The read request size is `capacity - len - 1`, deliberately reserving one byte so a null terminator can always be appended afterward, even though the data itself may be binary and the null terminator is mostly a convenience for treating text responses as C strings. `n == 0` is the

defining signal of a clean connection close (end-of-file on the socket) and is what allows this function to know when the peer is finished sending. `n < 0` indicates a genuine error (inspect `errno` for the cause).

The caller receives ownership of the heap-allocated buffer and is responsible for calling `free()` on it — exactly as documented in the header comment.

4.2.5 send_all

```
int send_all(int fd, const char *buf, size_t len) {
    size_t sent_total = 0;
    while (sent_total < len) {
        ssize_t n = send(fd, buf + sent_total, len - sent_total, 0);
        if (n < 0) {
            perror("send");
            return -1;
        }
        sent_total += (size_t)n;
    }
    return 0;
}
```

This is a direct implementation of the partial-write loop introduced in Section 2.5. `buf + sent_total` advances the pointer past the bytes already sent, and `len - sent_total` shrinks the remaining request size accordingly, so each iteration sends only what is left. The loop terminates once `sent_total` reaches `len`, guaranteeing (barring an error) that the entire buffer has been transmitted before the function returns.

Chapter 5

The Gopher Protocol

5.1 Historical Context

Gopher, defined in RFC 1436 (1991), predates the World Wide Web's mainstream adoption and was, for a period in the early 1990s, a leading protocol for distributing hierarchically-organized documents over the Internet. Its design goals were simplicity and low implementation cost, both of which make it an excellent teaching protocol today.

5.2 Protocol Overview

A Gopher exchange is a single request followed by a single response, after which the connection is closed:

1. The client opens a TCP connection to the server (traditionally port 70; this project uses port 7070 to avoid requiring root privileges, since ports below 1024 are typically restricted to privileged processes on Linux).
2. The client sends exactly one line of text, called a **selector**, terminated by `\r\n`. An empty selector (i.e. just `\r\n`) requests the server's root menu.
3. The server sends back the corresponding content as a raw byte stream.
4. The server closes the connection. This closure *is* the end-of-message signal — there is no length field or other explicit terminator required by the protocol (though menus conventionally end with a line containing only a period, as described below).

5.3 The Menu Format

When a selector refers to a directory-like resource rather than a plain document, the response is a **menu**: a list of further selectors the client can follow. Each line of a menu has the format:

```
<type><display_text>\t<selector>\t<host>\t<port>\r\n
```

- **type** is a single character indicating what kind of resource this entry is. This implementation uses `0` for a plain text file, `1` for a submenu (directory), and `3` for an error.
- **display_text** is the human-readable label a client should show to the user.
- **selector** is the string the client should send back if it chooses this entry.

- **host** and **port** tell the client where to connect to fetch this entry (allowing a single menu to reference resources on entirely different servers).

Fields within a line are separated by the **tab** character (`\t`), not spaces — a detail that is easy to overlook but essential for compliant parsing. A menu response is conventionally terminated by a line containing a single period (`.\r\n`).

5.4 Comparison Table: Gopher's Minimalism

Aspect	Gopher	HTTP (for comparison)
Request format	One line (the selector)	Request line + headers + blank line
Response format	Raw content	Status line + headers + blank line + body
End-of-message signal	Connection closure	`Content-Length` header
Status/error reporting	None (type `3` lines by convention)	Numeric status codes (200, 404, ...)
Methods	None — always "fetch this selector"	GET, POST, PUT, DELETE, ...

This comparison is expanded further once HTTP itself has been covered in Chapter 8.

Chapter 6

Line-by-Line: `gopher_server.c`

6.1 Includes and Constants

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "../common/socket_utils.h"

#define PORT 7070

static const char *root_dir = "gopher_root";
```

`PORT` is fixed at 7070 rather than the protocol's traditional port 70, because binding to any port below 1024 on Linux requires root (superuser) privileges. Using an unprivileged port lets the server run as a normal user. `root_dir` is declared `static` (file-local, not exported) and mutable, because `main()` may override it from a command-line argument, as seen later in this chapter.

6.2 `send_menu`: Building a Gopher Menu Response

```c
static void send_menu(int client_fd) {
    char response[8192] = {0};
    size_t offset = 0;

    offset += snprintf(response + offset, sizeof(response) - offset,
        "1Welcome Menu\tREADME\tlocalhost\t%d\r\n", PORT);
    offset += snprintf(response + offset, sizeof(response) - offset,
        "0About this server\tabout.txt\tlocalhost\t%d\r\n", PORT);
    offset += snprintf(response + offset, sizeof(response) - offset,
        ".\r\n");

    send_all(client_fd, response, offset);
}
```

This function constructs a Gopher menu (Section 5) directly in a fixed-size stack buffer. The `offset += snprintf(...)` pattern is a common idiom for incrementally building a string: `snprintf` returns the number of characters it *would have* written (excluding the null terminator), so accumulating these return values into `offset` tracks exactly how many bytes of `response`

are currently in use, and `sizeof(response) - offset` correctly shrinks the remaining available space on each call, preventing any individual `snprintf` call from overflowing the buffer.

Two hard-coded menu entries are produced: a directory-type entry (`1`) pointing at `README`, and a text-type entry (`0`) pointing at `about.txt`. Both entries advertise `localhost` and `PORT` as their host/port, meaning a compliant Gopher client following this menu would reconnect to this same server. The menu is terminated with the conventional `.\r\n` line, then transmitted in full using `send_all` (Chapter 4) rather than a raw `send` call, guaranteeing the entire buffer reaches the client even if the kernel's socket buffer requires multiple underlying writes.

6.3 `send_file`: Serving a Selector as a File

```
static void send_file(int client_fd, const char *selector) {
    char path[512];
    snprintf(path, sizeof(path), "%s/%s", root_dir, selector);

    FILE *f = fopen(path, "r");
    if (!f) {
        const char *msg = "3File not found\r\n.\r\n";
        send_all(client_fd, msg, strlen(msg));
        return;
    }
```

The requested `selector` is concatenated onto `root_dir` to build a filesystem path, then opened with the standard C library's `fopen`. If the file does not exist (or cannot be opened for any other reason — `fopen` does not distinguish), a Gopher-style error line is sent back using type `3`, per the convention introduced in Section 5.

This function performs no path-traversal validation on `selector`: a selector such as `../../../../etc/passwd` would be concatenated verbatim into `path` and, if readable by the server process, served without restriction. Contrast this with the HTTP server (Chapter 9), which explicitly checks for `".."` in the requested path. This is a deliberate simplification for the Gopher implementation, revisited in Chapter 14.

```
    char buf[4096];
    size_t n;
    while ((n = fread(buf, 1, sizeof(buf), f)) > 0) {
        send_all(client_fd, buf, n);
    }
    fclose(f);

    send_all(client_fd, ".\r\n", 3);
}
```

The file is streamed to the client in 4096-byte chunks rather than being read entirely into memory first. This is a deliberate memory-efficiency choice: it keeps the server's memory footprint bounded regardless of how large the requested file is, at the cost of one `fread`/`send_all` pair per chunk. After the file's contents are fully sent, a trailing `.\r\n` is appended. Strictly, RFC 1436 only requires this terminator for menu-type (type `1`) responses, but sending it unconditionally here is a simplification that keeps the code path uniform, at the minor cost of appending two extraneous bytes to plain-text file downloads.

6.4 `handle_client`: Parsing the Request and Dispatching

```c
static void handle_client(int client_fd) {
    char selector[512] = {0};
    ssize_t n = recv(client_fd, selector, sizeof(selector) - 1, 0);
    if (n <= 0) {
        close(client_fd);
        return;
    }
    selector[n] = '\0';
```

Because a Gopher request is always a single short line, this function reads it with one `recv` call rather than the general-purpose looping helpers from `socket_utils.c`. This is a reasonable simplification given the protocol's guarantee that requests are small, but it does mean that a pathological client sending its selector across multiple very small TCP segments could, in principle, have its request split across more than one `recv` call — a scenario this simplified handler does not account for. `n <= 0` handles both the "client disconnected immediately" (`n == 0`) and "error" (`n < 0`) cases identically, since in both situations there is no request to process.

```c
    char *cr = strpbrk(selector, "\r\n");
    if (cr) *cr = '\0';

    printf("[gopher] selector requested: \"%s\"\n", selector);
```

`strpbrk` searches `selector` for the *first occurrence of any character* in the given set — here, either `\r` or `\n`. If found, that character is overwritten with a null terminator, effectively trimming the trailing line ending sent by the client. This correctly handles clients that send either bare `\n` or the protocol-correct `\r\n`.

```c
    if (strlen(selector) == 0) {
        send_menu(client_fd);
    } else {
        send_file(client_fd, selector);
    }

    close(client_fd);
}
```

This is the entire dispatch logic of the server: an empty selector (after trimming) requests the root menu; anything else is treated as a filename to serve. The connection is unconditionally closed afterward, which is what makes this a compliant Gopher server — recall from Chapter 5 that connection closure *is* the protocol's end-of-message signal.

6.5 `main`: Startup and the Accept Loop

```c
int main(int argc, char *argv[]) {
    if (argc > 1) {
        root_dir = argv[1];
    }

    setvbuf(stdout, NULL, _IONBF, 0);
```

If a command-line argument is provided, it overrides the default `"gopher_root"` directory — this is what allows the server to be pointed at an arbitrary content directory, and is discussed further in Chapter 12 when explaining a real gotcha encountered while testing this project. `setvbuf(stdout, NULL, _IONBF, 0)` disables `stdout` buffering entirely. By default, C's standard I/O library buffers output differently depending on whether `stdout` is connected to an interactive terminal (line-buffered) or to a file/pipe (fully buffered). When a server is launched

in the background with its output redirected to a log file, full buffering means `printf` calls may sit in memory for a long time before actually being written — disabling buffering ensures log lines appear immediately, which matters both for live debugging and for the testing procedure documented in Chapter 13.

```
    int server_fd = create_server_socket(PORT, 10);
    if (server_fd < 0) {
        fprintf(stderr, "Failed to start server\n");
        return 1;
    }

    printf("Gopher server running on port %d (root: %s)\n", PORT, root_dir);
    printf("Try: printf '\\r\\n' | nc localhost %d\n", PORT);
```

The listening socket is created via the shared helper from Chapter 4, with a backlog of 10 pending connections. Startup diagnostics are printed, including a ready-to-use example command using `nc` (netcat) for manual testing.

```
    while (1) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);
        int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &
    addr_len);
        if (client_fd < 0) {
            perror("accept");
            continue;
        }

        printf("[gopher] New connection from %s\n", inet_ntoa(client_addr.
    sin_addr));
        handle_client(client_fd);
    }

    close(server_fd);
    return 0;
}
```

This is the server's main loop, and it runs forever. Each iteration blocks on `accept` until a client connects, at which point `client_addr` is populated with the connecting peer's address information. `inet_ntoa` converts the binary IP address into its familiar dotted-decimal string form (e.g. `"127.0.0.1"`) for the log message. `handle_client` is then called synchronously — meaning the server processes exactly one client at a time and cannot accept a second connection until the first has been fully handled and closed. This single-threaded, sequential design is a deliberate simplification; see Chapter 14 for a discussion of concurrency as a possible extension.

The final `close(server_fd)` and `return 0` are technically unreachable in normal operation, since the `while(1)` loop above never exits except via external termination (e.g. `Ctrl+C`, or the process being killed). They are included for completeness and to document what *would* happen were the loop ever to exit via a `break`.

Chapter 7

Line-by-Line: `gopher_client.c`

7.1 Overview

The Gopher client is deliberately short: it exists to demonstrate that the *same* shared utility functions used to build the server (Chapter 4) work identically from the client side, and to provide a real end-to-end counterpart for testing the server against something other than a Python script or `nc`.

7.2 Argument Parsing

```
int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(stderr, "Usage: %s <host> <port> [selector]\n", argv[0]);
        fprintf(stderr, "Example: %s localhost 7070 about.txt\n", argv[0]);
        return 1;
    }

    const char *host = argv[1];
    int port = atoi(argv[2]);
    const char *selector = (argc >= 4) ? argv[3] : "";
```

The client requires at least a host and a port; the selector is optional and defaults to the empty string, which (per the protocol, Chapter 5) requests the server's root menu. `argv[0]` is conventionally the program's own invocation name, which is why it appears in the usage message rather than a hard-coded string — this makes the message accurate even if the executable is renamed. Note that `atoi` performs no error checking: passing a non-numeric string as the port argument silently yields `0` rather than producing an error, a limitation discussed further in Chapter 14.

7.3 Connecting and Sending the Request

```
    int fd = connect_to_server(host, port);
    if (fd < 0) {
        fprintf(stderr, "Failed to connect to %s:%d\n", host, port);
        return 1;
    }

    char request[600];
    int req_len = snprintf(request, sizeof(request), "%s\r\n", selector);

```

```
    if (send_all(fd, request, (size_t)req_len) != 0) {
        fprintf(stderr, "Failed to send request\n");
        close(fd);
        return 1;
    }
```

`connect_to_server` (Chapter 4) handles hostname resolution and the TCP handshake in one call. The entire Gopher request — recall from Chapter 5 that this is simply the selector followed by `\r\n` — is built with a single `snprintf` into a stack buffer, then transmitted with `send_all` to guarantee it is delivered in full even if the underlying `send` call needs multiple attempts (Section 2.5).

7.4 Reading and Printing the Response

```
    size_t response_len = 0;
    char *response = read_until_close(fd, &response_len);
    close(fd);

    if (!response) {
        fprintf(stderr, "Failed to read response\n");
        return 1;
    }

    printf("%s", response);
    free(response);

    return 0;
}
```

Because Gopher signals the end of a response by closing the connection (Chapter 5), the client can simply call `read_until_close` and trust that it will return only once the server has finished sending and disconnected. The socket is closed immediately after reading, since nothing more will arrive on it. The response buffer, which was heap-allocated inside `read_until_close`, is printed directly and then explicitly freed — ownership of that allocation was documented as transferring to the caller in the header comment discussed in Chapter 4, and this is where that responsibility is discharged.

Note

This client prints the *raw* Gopher response verbatim, including the tab-separated menu syntax and trailing `.\r\n` terminator when the response is a menu. A "real" Gopher client would parse this structure and render a navigable menu; this implementation intentionally stops short of that, since the parsing itself does not exercise any new networking concept beyond what Chapter 5 already documents about the format.

Chapter 8

The HTTP Protocol (Implemented Subset)

8.1 Scope of This Implementation

Full HTTP/1.1 (RFC 9110–9112) is a large specification covering persistent connections, chunked transfer encoding, content negotiation, caching, range requests, and many other features. This project implements a deliberately narrow, but protocol-correct, subset:

- Only the **GET** method is supported; other methods receive a `405 Method Not Allowed` response.
- Every response includes `Connection: close`, meaning each TCP connection handles exactly one request/response pair — there is no persistent-connection (keep-alive) support.
- Content length is always communicated via an explicit `Content-Length` header; chunked transfer encoding is not implemented.
- A minimal set of status codes is used: `200 OK`, `400 Bad Request`, `404 Not Found`, and `405 Method Not Allowed`.

This scope was chosen to cover the concepts that most clearly extend beyond what Gopher already teaches (Chapter 5), without requiring an implementation of the full specification.

8.2 Request Format

```
GET /index.html HTTP/1.1
Host: localhost
Connection: close
```

An HTTP request consists of three parts, each terminated by `\r\n`:

1. **Request line**: `METHOD SP path SP HTTP-version`, e.g. `GET /index.html HTTP/1.1`.
2. **Headers**: zero or more `Name: value` lines.
3. **A blank line** (i.e. two consecutive `\r\n` sequences) marking the end of the headers and the start of any body.

For a `GET` request, there is normally no body, so the blank line effectively marks the end of the entire request.

8.3 Response Format

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 137
Connection: close

<html>...</html>
```

The response mirrors the request's structure:

1. **Status line**: `HTTP-version SP status-code SP reason-phrase`, e.g. `HTTP/1.1 200 OK`.
2. **Headers**, most importantly `Content-Type` (telling the client how to interpret the body) and `Content-Length` (telling the client exactly how many bytes of body follow).
3. **A blank line**.
4. **The body** — exactly `Content-Length` bytes.

8.4 The Central Technical Challenge: Framing

The hardest part of implementing HTTP correctly, and the concept this project is specifically designed to surface, is: *how does a reader know when the message is complete?*

Unlike Gopher, where the answer is simply "when the connection closes" (Chapter 5), HTTP as implemented here requires two distinct steps:

1. **Find the end of the headers.** This is always marked by the four-byte sequence `\r\n\r\n`. Everything before it is the status/request line plus headers; everything after it is the body.
2. **Determine the body length.** Parse the `Content-Length` header (if present) from the headers section, and read exactly that many bytes following the blank line.

This project's HTTP client (Chapter 10) implements both steps explicitly, since — unlike the server, which *generates* a well-formed response and therefore always knows its own body length — the client must *parse* an arbitrary incoming response without knowing its length in advance.

8.5 Comparison with Gopher

Aspect	HTTP (this implementation)	Gopher
Request structure	Request line + headers + blank line	Single selector line
Response structure	Status line + headers + blank line + body	Raw content
End-of-message signal	`Content-Length` header	Connection closure
Status reporting	Numeric codes (200, 400, 404, 405)	None (informal type-`3` error lines)
Methods	GET (others rejected with 405)	None — always "fetch"
Content typing	`Content-Type` header, guessed from file extension	None — client must infer from context

This table extends the one presented in Chapter 5, and is the clearest single summary of what implementing HTTP *after* Gopher actually adds, conceptually.

Chapter 9

Line-by-Line: `http_server.c`

9.1 Includes and Constants

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/stat.h>
#include "../common/socket_utils.h"

#define PORT 8080
#define MAX_REQUEST 8192

static const char *root_dir = "www";
```

The additional header here compared to `gopher_server.c` is `<sys/stat.h>`, needed for the `stat()` call used later to determine file sizes. `MAX_REQUEST` bounds the size of the buffer used to receive an incoming HTTP request; requests larger than 8192 bytes are not supported by this simplified implementation (a limitation discussed in Chapter 14).

9.2 `guess_content_type`: Mapping File Extensions to MIME Types

```c
static const char *guess_content_type(const char *path) {
    const char *ext = strrchr(path, '.');
    if (!ext) return "application/octet-stream";

    if (strcmp(ext, ".html") == 0 || strcmp(ext, ".htm") == 0) return "text/
    html";
    if (strcmp(ext, ".txt") == 0) return "text/plain";
    if (strcmp(ext, ".css") == 0) return "text/css";
    if (strcmp(ext, ".js") == 0) return "application/javascript";
    if (strcmp(ext, ".json") == 0) return "application/json";
    if (strcmp(ext, ".png") == 0) return "image/png";
    if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) return "image/
    jpeg";

    return "application/octet-stream";
}
```

`strrchr(path, '.')` finds the *last* occurrence of a period in the path, which correctly isolates the file extension even for filenames containing multiple periods (e.g. `archive.tar.gz` would match `.gz`). A small table of known extensions is checked in sequence; anything unrecognized falls back to `application/octet-stream`, the standard MIME type meaning "arbitrary binary data," which causes browsers to typically offer the file for download rather than attempting to render it. Real-world servers use a much larger table (often loaded from a `mime.types` file) covering hundreds of extensions; this function is a deliberately minimal illustration of the same idea.

9.3 parse_request_line: Extracting Method and Path

```
static int parse_request_line(const char *request, char *method, char *path)
    {
    const char *line_end = strstr(request, "\r\n");
    if (!line_end) return -1;

    int matched = sscanf(request, "%15s %255s", method, path);
    if (matched != 2) return -1;

    return 0;
}
```

This function extracts only the two fields the server actually needs from the request line (Chapter 8): the HTTP method and the requested path. The HTTP version field (e.g. `HTTP/1.1`) is intentionally not parsed or validated, since this implementation always responds with `HTTP/1.1` regardless of what the client requested — another deliberate scope reduction.

`sscanf` with the format `"%15s %255s"` reads two whitespace-delimited tokens. The numbers `15` and `255` are **field width limits**: they cap how many characters `sscanf` will write into `method` and `path` respectively, matching the sizes of the buffers the caller allocates for them (`16` and `256` bytes, leaving room for the null terminator). This is a critical safety measure — without explicit width limits, `%s` in `sscanf` will write as many characters as it finds before the next whitespace, with no regard for the destination buffer's actual size, which is a classic buffer-overflow vulnerability pattern. `matched != 2` catches malformed request lines where fewer than two tokens could be extracted (for example, an empty request).

9.4 send_response: The Core Response-Writing Function

```
static void send_response(int client_fd, int status_code, const char *
    status_text,
                          const char *content_type, const char *body, size_t
     body_len) {
    char header[512];
    int header_len = snprintf(header, sizeof(header),
        "HTTP/1.1 %d %s\r\n"
        "Content-Type: %s\r\n"
        "Content-Length: %zu\r\n"
        "Connection: close\r\n"
        "\r\n",
        status_code, status_text, content_type, body_len);

    send_all(client_fd, header, (size_t)header_len);
    if (body_len > 0) {
        send_all(client_fd, body, body_len);
    }
}
```

Every response the server sends — success or error — ultimately funnels through this single function, ensuring the response format (Chapter 8) is constructed consistently in exactly one place. The header block is built as a single `snprintf` call using adjacent string-literal concatenation (a standard C feature where consecutive quoted strings are automatically joined by the compiler), producing the status line followed by three headers and the mandatory blank line.

`Content-Length` is computed from `body_len`, which the caller must supply accurately — this is only possible because, as noted in Chapter 8, the *server* always knows its own body's exact size in advance (unlike the client, which must discover this by parsing an incoming response). `Connection: close` is sent unconditionally, correctly advertising to the client that this implementation does not support persistent connections. The header is sent first, then the body only if `body_len > 0` — this guard avoids calling `send_all` with a zero-length buffer for bodyless responses (such as the 500 error path seen later in this chapter).

9.5 Error Response Helpers

```
static void send_404(int client_fd) {
    const char *body = "<html><body><h1>404 Not Found</h1></body></html>";
    send_response(client_fd, 404, "Not Found", "text/html", body, strlen(body));
}

static void send_400(int client_fd) {
    const char *body = "<html><body><h1>400 Bad Request</h1></body></html>";
    send_response(client_fd, 400, "Bad Request", "text/html", body, strlen(body));
}

static void send_405(int client_fd) {
    const char *body = "<html><body><h1>405 Method Not Allowed</h1></body></html>";
    send_response(client_fd, 405, "Method Not Allowed", "text/html", body, strlen(body));
}
```

These three thin wrapper functions each hard-code a minimal HTML error page and delegate to `send_response`. Factoring them out keeps the call sites later in the file (inside `serve_file` and `handle_client`) short and self-documenting, e.g. a call site simply reads `send_404(client_fd);` rather than repeating the full response-construction logic at every point an error can occur.

9.6 `serve_file`: Reading and Serving a File from Disk

```
static void serve_file(int client_fd, const char *path) {
    char full_path[600];

    if (strcmp(path, "/") == 0) {
        snprintf(full_path, sizeof(full_path), "%s/index.html", root_dir);
    } else {
        snprintf(full_path, sizeof(full_path), "%s%s", root_dir, path);
    }
```

The requested URL path is translated into a filesystem path relative to `root_dir`. A request for `/` is special-cased to serve `index.html`, mirroring the conventional behavior of essentially all real web servers when a directory (rather than a specific file) is requested.

```
    if (strstr(path, "..") != NULL) {
        send_400(client_fd);
```

```
        return;
    }
```

This is the server's **path traversal defense**. Without it, a request such as `GET /../../etc/passwd` would have its path concatenated directly onto `root_dir`, potentially escaping the intended content directory entirely and exposing arbitrary files readable by the server process. The check here is intentionally simple — it rejects any path containing the literal substring `".."` — rather than performing full path canonicalization (resolving `.` and `..` segments and verifying the result still lies within `root_dir`, as a production server would do). Chapter 13 documents how this exact defense was verified to work, and specifically why testing it with `curl` initially gave a misleading result.

Notice that the `".."` check happens *after* `full_path` has already been constructed with `snprintf`, but the function correctly returns before that (unsafe) `full_path` is ever used to open a file. The check operates on the original, unmodified `path` argument, not on `full_path` — either would work here, but validating the raw untrusted input directly is the clearer pattern to follow in general.

```
    FILE *f = fopen(full_path, "rb");
    if (!f) {
        send_404(client_fd);
        return;
    }

    struct stat st;
    if (stat(full_path, &st) != 0) {
        fclose(f);
        send_404(client_fd);
        return;
    }
```

The file is opened in binary mode (`"rb"`) — on POSIX systems this is equivalent to text mode, but specifying it explicitly is good practice for portability and signals clear intent, since the server may serve non-text content (images, per `guess_content_type`). `stat()` retrieves file metadata, most importantly `st.st_size`, the exact file size in bytes, which is required to compute `Content-Length` correctly *before* any data is sent (recall from Chapter 8 that the header block, including `Content-Length`, must precede the body).

```
    char *body = malloc((size_t)st.st_size);
    if (!body) {
        fclose(f);
        send_response(client_fd, 500, "Internal Server Error", "text/plain",
    "", 0);
        return;
    }

    size_t read_bytes = fread(body, 1, (size_t)st.st_size, f);
    fclose(f);

    send_response(client_fd, 200, "OK", guess_content_type(full_path), body,
    read_bytes);
    free(body);
}
```

Unlike the Gopher server (Chapter 6), which streams file content in fixed-size chunks, this HTTP server reads the *entire* file into a single heap-allocated buffer before sending anything. This is

a direct consequence of the framing requirement discussed in Chapter 8: since `Content-Length` must be sent in the header *before* the body, and the header is built by `send_response` as a single unit, the simplest correct implementation determines the total size up front (via `stat`) and reads it all before calling `send_response`. This trades memory efficiency for implementation simplicity; a streaming HTTP implementation is possible (send headers first, then stream the body separately) but adds complexity intentionally avoided here, and is noted as a possible extension in Chapter 14.

`read_bytes` (the value actually returned by `fread`) is used for `Content-Length`, rather than `st.st_size`, as a defensive measure: in the (rare) case these values differ, the response header will always accurately describe what was actually placed in the buffer and subsequently sent.

9.7 `handle_client`: Reading and Dispatching a Request

```
static void handle_client(int client_fd) {
    char request[MAX_REQUEST] = {0};
    ssize_t n = recv(client_fd, request, sizeof(request) - 1, 0);
    if (n <= 0) {
        close(client_fd);
        return;
    }
    request[n] = '\0';
```

Similarly to the Gopher server's `handle_client`, this function reads the request with a single `recv` call rather than looping until a complete request is confirmed present. For the small, simple GET requests this server and its accompanying client generate, the request reliably arrives in one TCP segment on the loopback interface, but a fully robust implementation would need to handle the possibility of a request line and headers being split across multiple `recv` calls, analogous to the framing logic the HTTP *client* implements in Chapter 10.

```
    char method[16] = {0};
    char path[256] = {0};

    if (parse_request_line(request, method, path) != 0) {
        send_400(client_fd);
        close(client_fd);
        return;
    }

    printf("[http] %s %s\n", method, path);

    if (strcmp(method, "GET") != 0) {
        send_405(client_fd);
        close(client_fd);
        return;
    }

    serve_file(client_fd, path);
    close(client_fd);
}
```

The dispatch logic is straightforward: a malformed request line yields `400 Bad Request`; a well-formed request line with a method other than `GET` yields `405 Method Not Allowed`; and a well-formed `GET` request is handed to `serve_file`. Every path through this function ends in `close(client_fd)`, consistent with the `Connection: close` header always sent — the server never attempts to reuse a connection for a second request.

9.8 main: Startup and the Accept Loop

```c
int main(int argc, char *argv[]) {
    if (argc > 1) {
        root_dir = argv[1];
    }

    setvbuf(stdout, NULL, _IONBF, 0);

    int server_fd = create_server_socket(PORT, 10);
    if (server_fd < 0) {
        fprintf(stderr, "Failed to start server\n");
        return 1;
    }

    printf("HTTP server running on port %d (root: %s)\n", PORT, root_dir);
    printf("Try: curl http://localhost:%d/\n", PORT);

    while (1) {
        struct sockaddr_in client_addr;
        socklen_t addr_len = sizeof(client_addr);
        int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &
    addr_len);
        if (client_fd < 0) {
            perror("accept");
            continue;
        }

        printf("[http] New connection from %s\n", inet_ntoa(client_addr.
    sin_addr));
        handle_client(client_fd);
    }

    close(server_fd);
    return 0;
}
```

This function is structurally identical to the Gopher server's `main` (Chapter 6): optionally override `root_dir` from `argv[1]`, disable `stdout` buffering, create the listening socket, and loop forever accepting and synchronously handling one client connection at a time. The deliberate structural symmetry between the two servers' `main` functions is intended to make the *differences* between the two protocols — concentrated entirely in `handle_client` and its helpers — as visible as possible.

Chapter 10

Line-by-Line: `http_client.c`

10.1 Why the Client Is More Complex Than the Server

At first glance it may seem surprising that the HTTP *client* requires more intricate parsing logic than the server. The reason is fundamental, and was previewed in Chapter 8 (Section 8.4): the server *generates* every response it sends, so it always knows the exact body length in advance (via `stat()`, Chapter 9). The client, by contrast, receives an arbitrary byte stream from the network and must *discover* where the headers end and how many body bytes follow, using only the data itself. This chapter's two static helper functions exist specifically to solve that discovery problem.

10.2 Includes

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include "../common/socket_utils.h"
```

The only addition compared to the other three files in this project is `<ctype.h>`, needed for `tolower()` in `parse_content_length`, discussed below.

10.3 `find_headers_end`: Locating the Header/Body Boundary

```
static char *find_headers_end(char *buf, size_t len) {
    for (size_t i = 0; i + 3 < len; i++) {
        if (buf[i] == '\r' && buf[i+1] == '\n' && buf[i+2] == '\r' && buf[i
    +3] == '\n') {
            return buf + i;
        }
    }
    return NULL;
}
```

This function performs a linear scan of the raw response buffer looking for the four-byte sequence `\r\n\r\n`, which — as established in Chapter 8 — unambiguously marks the boundary between the headers section and the body of an HTTP message. The loop condition `i + 3 < len` (rather than `i < len`) is a bounds-safety detail: since the check inside the loop reads `buf[i+3]`, the

loop must stop early enough that `i+3` never indexes past the end of the buffer, which would be undefined behavior (reading memory outside the allocated region).

Note

This function operates on raw bytes, not a null-terminated C string function like `strstr`. This matters because an HTTP response body can legitimately contain binary data (e.g. an image) that includes null bytes or any other byte value — using a string function that stops at the first `'\0'` could silently truncate the search. Scanning byte-by-byte with an explicit length, as done here, is correct regardless of the body's content.

If no such sequence is found, the function returns `NULL`, which the caller (`main`, below) treats as a fatal parsing error — a well-formed HTTP response, by definition, always contains this boundary.

10.4 `parse_content_length`: Extracting a Header Value

```
static long parse_content_length(const char *headers) {
    const char *p = headers;
    size_t target_len = strlen("content-length:");

    while (*p) {
        size_t i = 0;
        while (i < target_len && p[i] &&
               tolower((unsigned char)p[i]) == "content-length:"[i]) {
            i++;
        }
        if (i == target_len) {
            return atol(p + target_len);
        }
        p++;
    }
    return -1;
}
```

This function searches for the `Content-Length:` header within the (null-terminated) headers text and, if found, parses the numeric value that follows it. It is written as a manual case-insensitive substring search rather than using `strstr` directly, because HTTP header names are defined by the specification to be **case-insensitive** — a server could legally send `content-length`, `Content-Length`, or even `CONTENT-LENGTH`, and a correct client must recognize all of them identically.

The outer `while (*p)` loop advances one character at a time through the header text, treating each position as a candidate starting point. At each candidate position, the inner loop compares up to `target_len` characters, converting each byte from the input to lowercase with `tolower()` before comparing it against the corresponding (already-lowercase) character of the literal `"content-length:"`. If the inner loop runs to completion (`i == target_len`), a full case-insensitive match has been found at this position, and `atol` parses the digits immediately following the colon into a `long`. If no match is found at the current position, the outer loop simply advances `p` by one and tries again.

`tolower()` technically expects an `int` argument that is either `EOF` or representable as an `unsigned char`; passing a plain (possibly signed) `char` directly can trigger undefined behavior on platforms where `char` is signed and the byte value is negative (e.g. certain non-ASCII bytes). The explicit cast `(unsigned char)p[i]` guards against this and is considered best practice whenever calling `tolower`, `toupper`, or similar `<ctype.h>` functions on raw `char` data.

If the header is never found (for instance, in a response with an empty body and no `Content-Length` at all), the function returns `-1`, a sentinel value the caller checks for before printing any diagnostic about the declared length.

10.5 `main`: Request Construction

```c
int main(int argc, char *argv[]) {
    if (argc < 4) {
        fprintf(stderr, "Usage: %s <host> <port> <path>\n", argv[0]);
        fprintf(stderr, "Example: %s localhost 8080 /\n", argv[0]);
        return 1;
    }

    const char *host = argv[1];
    int port = atoi(argv[2]);
    const char *path = argv[3];

    int fd = connect_to_server(host, port);
    if (fd < 0) {
        fprintf(stderr, "Failed to connect to %s:%d\n", host, port);
        return 1;
    }

    char request[1024];
    int req_len = snprintf(request, sizeof(request),
        "GET %s HTTP/1.1\r\n"
        "Host: %s\r\n"
        "Connection: close\r\n"
        "\r\n",
        path, host);

    if (send_all(fd, request, (size_t)req_len) != 0) {
        fprintf(stderr, "Failed to send request\n");
        close(fd);
        return 1;
    }
```

Unlike the Gopher client (Chapter 7), which requires host, port, and an *optional* selector, this client requires all three arguments (`argc < 4`), since an HTTP request without a path is not meaningful in the same way an empty Gopher selector is. The request is built as a single well-formed HTTP/1.1 GET request (per the format defined in Chapter 8), including a mandatory `Host` header — required by the HTTP/1.1 specification for every request, since a single server process may serve multiple domain names on the same IP address — and a `Connection: close` header, explicitly telling the server this client does not support persistent connections either, matching the server's own behavior (Chapter 9).

10.6 `main`: Reading and Splitting the Response

```
    size_t total_len = 0;
    char *response = read_until_close(fd, &total_len);
    close(fd);

    if (!response) {
        fprintf(stderr, "Failed to read response\n");
        return 1;
    }
```

Note

This client reuses `read_until_close` — the same function the Gopher client uses — rather than implementing the more general "read headers, then read exactly `Content-Length` more bytes" pattern described in Chapter 8. This works correctly here *only because* the request explicitly sent `Connection: close`, and this project's server (Chapter 9) honors that header by always closing the connection after one response. If this client needed to talk to a real-world, keep-alive-capable server, it would need to switch to reading incrementally and stopping as soon as the declared `Content-Length` bytes have been received, rather than waiting for the connection to close (which, under keep-alive, it might not, for a long time).

```
    char *headers_end = find_headers_end(response, total_len);
    if (!headers_end) {
        fprintf(stderr, "Invalid response: no clear end of headers\n");
        free(response);
        return 1;
    }

    size_t headers_len = (size_t)(headers_end - response);
    char *body_start = headers_end + 4;
    size_t body_len_from_socket = total_len - headers_len - 4;
```

Once the full raw response has been read into `response`, `find_headers_end` locates the `\r\n\r\n` boundary. Pointer subtraction (`headers_end - response`) yields `headers_len`, the number of bytes occupied by the status line and headers. `body_start` is computed by advancing four bytes past the boundary pointer (skipping over the `\r\n\r\n` sequence itself), and `body_len_from_socket` is derived arithmetically as "everything else": the total bytes received, minus the header bytes, minus the four boundary bytes.

10.7 `main`: Printing and the Length Cross-Check

```
    printf("--- Headers ---\n");
    fwrite(response, 1, headers_len, stdout);
    printf("\n\n");

    long declared_len = parse_content_length(response);
    if (declared_len >= 0) {
        printf("--- Declared Content-Length: %ld, actually received: %zu ---\
    n",
               declared_len, body_len_from_socket);
    }

    printf("--- Body ---\n");
    fwrite(body_start, 1, body_len_from_socket, stdout);
    printf("\n");

    free(response);
    return 0;
}
```

`fwrite` is used instead of `printf("%s", ...)` to print the headers and body sections, because both are byte ranges of a known, explicit length rather than null-terminated strings — again correctly handling the possibility of embedded null bytes in binary content, consistent with the byte-oriented approach taken throughout this file.

The diagnostic line comparing `declared_len` (parsed from the `Content-Length` header itself) against `body_len_from_socket` (independently computed from the raw byte count actually received over the socket) is not strictly necessary for the client to function, but is included deliberately as a pedagogical cross-check: when the server is implemented correctly, these two numbers will always agree, which is a direct, observable confirmation that the framing logic on both ends of the connection is behaving as the protocol requires. Chapter 13 shows this cross-check succeeding in practice, and discusses what a discrepancy between the two numbers would indicate about a bug in either the server or the client.

Finally, `free(response)` releases the buffer allocated inside `read_until_close`, matching the ownership contract documented in Chapter 4.

Chapter 11

The Build System: `Makefile`

11.1 Full Listing

```
CC = gcc
CFLAGS = -Wall -Wextra -g -std=c11

COMMON = common/socket_utils.c

.PHONY: all clean run-gopher-server run-http-server

all: gopher_server gopher_client http_server http_client

gopher_server: gopher/gopher_server.c $(COMMON)
    $(CC) $(CFLAGS) -o gopher_server gopher/gopher_server.c $(COMMON)

gopher_client: gopher/gopher_client.c $(COMMON)
    $(CC) $(CFLAGS) -o gopher_client gopher/gopher_client.c $(COMMON)

http_server: http/http_server.c $(COMMON)
    $(CC) $(CFLAGS) -o http_server http/http_server.c $(COMMON)

http_client: http/http_client.c $(COMMON)
    $(CC) $(CFLAGS) -o http_client http/http_client.c $(COMMON)

run-gopher-server: gopher_server
    ./gopher_server

run-http-server: http_server
    ./http_server

clean:
    rm -f gopher_server gopher_client http_server http_client
```

11.2 Variables

```
CC = gcc
CFLAGS = -Wall -Wextra -g -std=c11
COMMON = common/socket_utils.c
```

`CC` names the compiler to invoke; keeping it as a variable (rather than hard-coding `gcc` in every rule) means a reader could override it at invocation time, e.g. `make CC=clang`, without editing the file. `CFLAGS` defines the compiler flags used uniformly across every target:

Flag	Meaning
`-Wall`	Enable a broad set of common compiler warnings.
`-Wextra`	Enable additional warnings not covered by `-Wall`.
`-g`	Include debugging symbols in the binary, enabling use of tools like `gdb`.
`-std=c11`	Compile strictly against the C11 language standard (see also the discussion of `_POSIX_C_SOURCE` in Chapter 4, needed precisely because this flag disables POSIX extensions by default).

`COMMON` stores the path to the shared utility source file, since it appears as a dependency and compilation input in all four executable targets — defining it once avoids repeating the literal path four times and centralizes any future rename.

11.3 The `.PHONY` Declaration

```
.PHONY: all clean run-gopher-server run-http-server
```

By default, `make` treats every target name as though it *might* correspond to a file it should check the existence and modification time of. `.PHONY` tells `make` that the listed targets are not actual files, so it should always execute their recipes when invoked, regardless of whether a file with that name happens to exist in the working directory. This matters concretely here: without declaring `clean` as phony, if a file literally named `clean` ever existed in the project directory, `make clean` would silently do nothing, since `make` would see the (unrelated) file already exists and consider the target up to date.

11.4 The `all` Target and Dependency-Based Builds

```
all: gopher_server gopher_client http_server http_client
```

`all` has no recipe of its own; it exists purely to depend on all four executables. Running `make` with no arguments builds the first target defined in the file by convention, which is `all` here, causing every executable to be built (or rebuilt, if sources changed) in one command.

```
gopher_server: gopher/gopher_server.c $(COMMON)
    $(CC) $(CFLAGS) -o gopher_server gopher/gopher_server.c $(COMMON)
```

Each executable target follows the same pattern: the target name is followed by its *prerequisites* (the source files it depends on), and the indented line beneath is the *recipe* — the shell command `make` runs to actually produce the target. `make`'s core feature is that it compares the modification timestamp of the target file against each of its prerequisites, and only re-runs the recipe if the target is missing or older than at least one prerequisite. In practice, this means editing only `gopher_server.c` and running `make all` will recompile `gopher_server` but correctly skip recompiling `http_server`, `http_client`, and `gopher_client`, since their own source files are unchanged — a significant time saver as a project grows, even though the benefit is modest at this project's small scale.

Every recipe line in a `Makefile` **must** begin with a literal tab character, not spaces. This is a long-standing and frequently-encountered `make` pitfall: a recipe line indented with spaces produces the error `missing separator`, and many text editors silently convert leading tabs to spaces unless configured not to. If cloning or hand-copying this `Makefile`, this detail is worth double-checking if `make` reports that error.

11.5 Convenience Targets: `run-gopher-server` and `run-http-server`

```
run-gopher-server: gopher_server
    ./gopher_server

run-http-server: http_server
    ./http_server
```

These targets depend on the corresponding executable, so `make` automatically builds it first if necessary (or if source files have changed since the last build), and then immediately runs it in the foreground. This is a convenience for quick local testing but has a practical limitation: since the recipe simply runs `./gopher_server` with no arguments, the server always uses its compiled-in default content directory (`gopher_root` or `www`) and therefore must be invoked with `make` from *within* the corresponding protocol subdirectory to find that directory correctly — the precise gotcha documented in detail in Chapter 12.

11.6 The `clean` Target

```
clean:
    rm -f gopher_server gopher_client http_server http_client
```

`clean` has no prerequisites, so running `make clean` always executes its recipe (subject to the `.PHONY` rule above, ensuring it is never skipped because a same-named file happens to exist). `-f` suppresses any error `rm` would otherwise produce if one of the listed files does not currently exist, making it safe to run `make clean` even on a directory that has never been built.

Chapter 12

Running on Linux

This chapter is a practical, step-by-step guide to building and running this project on a Linux system, followed by a documented account of the real problems encountered while first bringing this project up, and how each was diagnosed and fixed. These are included deliberately: they are the kind of issue any reader is likely to hit when running similar code themselves.

12.1 Installing Prerequisites

On Debian/Ubuntu-based distributions:

```
sudo apt-get update
sudo apt-get install -y build-essential
```

On Arch Linux:

```
sudo pacman -S base-devel
```

Either command installs `gcc`, `make`, and the standard set of tools needed to build C projects. No third-party libraries are required — this project depends only on the C standard library and the POSIX sockets API, both provided by the base system.

12.2 Building

```
cd Protocols_Implementation
make all
```

This produces four executables in the repository root: `gopher_server`, `gopher_client`, `http_server`, `http_client`. Expect no warnings given the `-Wall -Wextra` flags in the `Makefile` (Chapter 11); any warning that does appear is worth investigating before proceeding.

12.3 Running the Gopher Server and Client

Terminal 1 — start the server, run from *inside* the `gopher/` directory:

```
cd gopher
../gopher_server gopher_root
```

Terminal 2 — run the client from the repository root:

```
cd Protocols_Implementation
./gopher_client localhost 7070              # root menu
./gopher_client localhost 7070 about.txt    # a specific file
```

12.4 Running the HTTP Server and Client

Terminal 1:

```
cd http
../http_server www
```

Terminal 2:

```
cd Protocols_Implementation
./http_client localhost 8080 /
./http_client localhost 8080 /about.txt

# Or with curl / a browser:
curl http://localhost:8080/
# http://localhost:8080/ in any browser
```

Stopping either server is done with `Ctrl+C` in its terminal.

12.5 Documented Issues Encountered During Development

The following problems were all encountered, diagnosed, and fixed while originally building and testing this project. They are recorded here because each illustrates a general Linux/C development lesson, not just a one-off mistake.

12.5.1 Issue 1: `getaddrinfo` Undeclared Under `-std=c11`

Symptom. Compiling `socket_utils.c` failed with the compiler unable to find declarations for `getaddrinfo` and related functions.

Cause. The `Makefile` compiles with `-std=c11`, which restricts visible library declarations to the strict ISO C standard. POSIX-specific functions like `getaddrinfo` are hidden by default under this flag.

Fix. Add `#define _POSIX_C_SOURCE 200809L` as the very first line of `socket_utils.c`, before any `#include` directive. This is discussed in depth in Chapter 4.

12.5.2 Issue 2: Background Server Processes Disappearing

Symptom. Starting a server with a trailing `&` to run it in the background, then issuing further commands in what appeared to be the same shell session, resulted in the server process no longer running — connection attempts failed with `Connection refused`.

Cause. In some automated/scripted execution environments (and in some shell configurations), each command may execute in its own subshell or process group, meaning a plain backgrounded (`&`) child process gets terminated when its parent shell instance exits at the end of that command, well before a subsequent command has a chance to talk to it.

Fix. Launch the server with `setsid`, which starts the process in a new session, detached from the controlling terminal/shell's process group:

```
setsid ./gopher_server gopher_root > server.log 2>&1 < /dev/null &
```

This makes the server persist independently of whichever shell invocation launched it. In an interactive terminal session (the normal case for a human running this project directly), a simple `&` is sufficient, since the terminal session itself persists across commands; `setsid` specifically matters for scripted or automated launches.

12.5.3 Issue 3: Server Output Not Appearing When Redirected to a File

Symptom. After solving Issue 2, the server's log file remained empty for a noticeable delay even though the server was confirmed to be running and successfully handling requests.

Cause. The C standard library buffers `stdout` differently depending on whether it is connected to an interactive terminal (line-buffered: flushed after every newline) or to a file/pipe (fully buffered: flushed only when the internal buffer fills, or on program exit). Redirecting a server's output to a log file switches it into full buffering, so `printf` calls can sit in memory for a long time before actually being written to disk.

Fix. Call `setvbuf(stdout, NULL, _IONBF, 0);` near the start of `main()`, disabling buffering entirely so every `printf` call is flushed immediately. This is why both server programs include this call, as noted in Chapters 6 and 9.

12.5.4 Issue 4: Gopher Server Returning "File Not Found" for Files That Exist

Symptom. The Gopher server correctly served its root menu, but every attempt to fetch an actual file (e.g. `about.txt`) returned the type-3 "File not found" error, even though the file was verifiably present on disk.

Cause. The server initially hard-coded its content directory as the relative path `"./gopher_root"`. A relative path is always resolved against the process's **current working directory** at the time the file is opened — *not* the directory containing the source file or the executable. The server had been launched from the repository root (`Protocols_Implementation/`), not from inside `gopher/`, so `./gopher_root` resolved to a nonexistent path relative to the root directory, rather than the intended `gopher/gopher_root`.

Fix. Two changes were made together: (1) the hard-coded path was replaced with a variable, `root_dir`, defaulting to the bare name `"gopher_root"` but overridable via `argv[1]`; and (2) the documented convention (see the running instructions earlier in this chapter) is to always launch the server from *inside* its protocol subdirectory, explicitly passing the content directory name as an argument. This makes the dependency on the working directory explicit and controllable, rather than an invisible assumption baked into the binary.

This is a general lesson, not specific to this project: any C program that opens files using relative paths is implicitly depending on whatever directory it happens to be launched from. When debugging an unexpected "file not found" error in any C (or, indeed, any) program, checking the process's actual current working directory (on Linux, visible via `readlink /proc/<pid>/cwd`) is one of the first things worth verifying.

12.5.5 Issue 5: `curl` Appearing to Defeat the Path-Traversal Defense

Symptom. A request for `curl "http://localhost:8080/../etc/passwd"` was expected to trigger the server's `400 Bad Request` path-traversal check (Chapter 9), but instead returned a plain `404 Not Found`.

Cause. This was not a bug in the server. `curl` performs URL **normalization** on the client side before ever sending the request: it collapses `..` segments in the path locally, so `/../etc/passwd` was transmitted over the wire as simply `/etc/passwd` — a path that legitimately contains no `".."` substring by the time the server ever sees it, and therefore correctly does not trigger the traversal check (it instead correctly reports 404, since no such file exists under `www/`).

Resolution. This was confirmed, not "fixed," by sending a raw, unnormalized request directly over a socket (bypassing `curl`'s normalization) using a short Python script, which did correctly trigger the server's `400` response. This is documented further, with the exact verification method, in Chapter 13.

Note

The lesson here generalizes: a server's security checks must be verified against the *literal bytes* a client could send, not only against the behavior of one particular well-behaved client library. Different HTTP clients (browsers, `curl`, hand-written scripts, deliberately malicious tools) make different choices about normalization, and a server cannot assume any particular client-side sanitization has already happened before its request arrives.

Chapter 13

Testing and Verification

This chapter documents how each component of this project was actually verified to behave correctly. All results shown reflect real test runs performed against the compiled binaries, not hypothetical output.

13.1 Gopher Server: Manual Verification

With the server running (`./gopher_server gopher_root` from within `gopher/`), the root menu was requested by sending an empty selector:

```
$ python3 -c "
import socket
s = socket.create_connection(('localhost', 7070))
s.sendall(b'\r\n')
print(s.recv(4096).decode())
"
1Welcome Menu   README  localhost   7070
0About this server  about.txt   localhost   7070
.
```

This confirms the menu format matches the specification described in Chapter 5: tab-separated fields, correct type characters, and the terminating `.` line.

Requesting an existing file and a nonexistent file were both verified:

```
$ ./gopher_client localhost 7070 about.txt
This is a sample Gopher file served over TCP.
.

$ ./gopher_client localhost 7070 nope.txt
3File not found
.
```

The server's own log confirmed each request was received and dispatched correctly:

```
[gopher] New connection from 127.0.0.1
[gopher] selector requested: "about.txt"
[gopher] New connection from 127.0.0.1
[gopher] selector requested: "nope.txt"
```

13.2 Gopher Client Against the Gopher Server: End-to-End

With the compiled `gopher_client` (rather than an ad-hoc Python script) run against the compiled `gopher_server`, all cases behaved identically to the manual tests above, and additionally the client's own argument-validation was exercised:

```
$ ./gopher_client
Usage: ./gopher_client <host> <port> [selector]
Example: ./gopher_client localhost 7070 about.txt
$ echo $?
1
```

This confirms the client correctly detects missing required arguments, prints a usage message to `stderr`, and exits with a nonzero status code — the standard convention for signaling failure to calling shell scripts or automated tooling.

13.3 HTTP Server: Status Codes and Headers

With the server running (`./http_server www` from within `http/`), each supported response path was exercised with `curl -i` (which prints response headers):

```
$ curl -s -i http://localhost:8080/
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 175
Connection: close

<!DOCTYPE html>
...

$ curl -s -i http://localhost:8080/notfound.html
HTTP/1.1 404 Not Found
Content-Type: text/html
Content-Length: 48
Connection: close

<html><body><h1>404 Not Found</h1></body></html>

$ curl -s -i -X POST http://localhost:8080/
HTTP/1.1 405 Method Not Allowed
...
```

Each response's `Content-Length` header was confirmed to match the actual byte length of the body that followed, verifying the header-writing logic in `send_response` (Chapter 9).

13.4 The Path-Traversal Test: A Worked Example of a Misleading Result

This test is documented in detail because the first attempt produced a result that looked like a failure but was not.

First attempt, using `curl`:

```
$ curl -s -i "http://localhost:8080/../etc/passwd"
HTTP/1.1 404 Not Found
...
```

At first glance, this looks wrong: the expectation was a `400 Bad Request` from the path-traversal check in `serve_file` (Chapter 9). As explained in Issue 5 of Chapter 12, the actual cause is that `curl` normalizes `..` segments out of the URL *before* sending the request, so the server never actually receives a path containing `".."`.

Second attempt, bypassing `curl`'s normalization by sending a raw request directly over a socket:

```
$ python3 -c "
import socket
s = socket.create_connection(('localhost', 8080), timeout=3)
s.sendall(b'GET /../../etc/passwd HTTP/1.1\r\nHost: localhost\r\n\r\n')
print(s.recv(4096).decode())
"
HTTP/1.1 400 Bad Request
Content-Type: text/html
Content-Length: 50
Connection: close

<html><body><h1>400 Bad Request</h1></body></html>
```

This confirms the defense works correctly against a request that actually contains the literal `".."` substring the check is designed to catch. The server's log for this session shows both requests clearly, including the un-normalized path exactly as sent:

```
[http] GET /etc/passwd
[http] GET /../../etc/passwd
```

Note

This worked example is preserved here deliberately, rather than only showing the final passing test, because the debugging process — recognizing that an apparently-failing test was actually a client-side artifact, and constructing a more direct test to isolate the real behavior — is itself a core skill this project is meant to build.

13.5 HTTP Client: Framing Verification

The compiled `http_client` was run against the compiled `http_server` for each response type, with particular attention to the declared-vs-received length cross-check described in Chapter 10:

```
$ ./http_client localhost 8080 /
--- Headers ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 175
Connection: close

--- Declared Content-Length: 175, actually received: 175 ---
--- Body ---
<!DOCTYPE html>
...

$ ./http_client localhost 8080 /about.txt
...
--- Declared Content-Length: 24, actually received: 24 ---
...

$ ./http_client localhost 8080 /notfound.html
...
```

```
--- Declared Content-Length: 48, actually received: 48 ---
...
```

In every case, the `Content-Length` value parsed from the response headers exactly matched the number of body bytes independently computed from the raw socket data (`body_len_from_socket` in `http_client.c`). This is the concrete, observable confirmation that the framing logic described conceptually in Chapter 8 (Section 8.4) and implemented in Chapters 9 and 10 is correct on both ends of the connection.

13.6 Summary of Verified Behavior

Behavior	Result
Gopher root menu format	Correct (tab-separated, correct types)
Gopher file retrieval (existing file)	Correct
Gopher file retrieval (missing file)	Correct (type-3 error)
Gopher client argument validation	Correct (usage message, exit code 1)
HTTP 200 response with correct headers	Correct
HTTP 404 for missing file	Correct
HTTP 405 for non-GET method	Correct
HTTP path-traversal defense (raw request)	Correct (400 Bad Request)
HTTP client Content-Length cross-check	Correct in all tested cases

Chapter 14

Limitations and Future Work

This project is a deliberately scoped teaching implementation, not a production-grade server or client. This chapter records, honestly and specifically, what has been intentionally left out, so that a reader extending this code understands exactly what remains to be done and why each omission was made.

14.1 Concurrency

Both servers handle exactly one client connection at a time: the `accept()` loop in `main()` (Chapters 6 and 9) calls `handle_client()` synchronously, and does not call `accept()` again until that call returns. A second client attempting to connect while the first is being served will simply wait in the kernel's connection backlog queue (sized by the `backlog` argument passed to `listen()`) until the first client's request is fully handled.

For the small, fast, local file-serving workloads this project targets, this is rarely noticeable in practice. A production server needs some form of concurrency to serve multiple clients simultaneously. The two classical approaches on POSIX systems are:

- **Process-per-connection**, using `fork()` immediately after `accept()` returns, letting the child process handle that one client while the parent immediately loops back to `accept()` the next.
- **Thread-per-connection**, using `pthread_create()` similarly, which avoids the memory overhead of a full process fork at the cost of requiring care around any shared mutable state.

Either would be a natural first extension to this codebase, and would require no changes to `socket_utils.c` or the protocol-parsing logic — only to the accept loop in each server's `main()`.

14.2 HTTP Persistent Connections (Keep-Alive)

Every response includes `Connection: close` (Chapter 9), and the client always closes its connection after one exchange (Chapter 10). Real HTTP/1.1 servers default to *keep-alive*: reusing a single TCP connection for multiple sequential requests, which avoids the overhead of a new TCP handshake per request. Supporting this would require the server's `handle_client` to loop, reading and responding to multiple requests per connection until the client explicitly closes it or a timeout elapses, and would require the client to read responses using the length-then-body strategy described in Chapter 8 rather than the simpler "read until close" approach it currently reuses from the Gopher client (as noted explicitly in Chapter 10).

14.3 Chunked Transfer Encoding

This implementation always sends a fully-known `Content-Length`, computed by reading an entire file into memory before responding (Chapter 9). Real HTTP servers often need to stream responses whose total length is not known in advance (e.g. dynamically generated content), for which HTTP defines **chunked transfer encoding** as an alternative framing mechanism. Implementing it would extend the framing discussion in Chapter 8 with a second, more complex, self-delimiting format.

14.4 Robustness of Request Parsing

Both servers read an incoming request with a single `recv()` call (Chapters 6 and 9), rather than looping until a complete request is confirmed present, as the general principle in Section 2.5 would suggest for full robustness. This works reliably for the small requests generated by this project's own clients on the loopback interface, but is not guaranteed correct in general — a sufficiently large or awkwardly-fragmented request could in principle arrive split across multiple TCP segments and thus multiple `recv()` calls. A fully robust server would apply the same "read headers, find the boundary, then read exactly `Content-Length` more" logic on the *server* side that this project's HTTP *client* already implements (Chapter 10).

14.5 Gopher Path-Traversal Protection

As noted explicitly in Chapter 6, `send_file` in `gopher_server.c` performs no validation of the requested selector before using it to construct a filesystem path, unlike the HTTP server's deliberate (if simple) `".."` check. Bringing the same defense to the Gopher server — or, better, replacing both implementations' substring check with proper path canonicalization and a containment check against `root_dir` — would be a natural security-hardening exercise.

14.6 Input Validation

Command-line port arguments are parsed with `atoi()` in both clients (Chapters 7 and 10), which silently returns `0` for non-numeric input rather than reporting an error. Using `strtol()` with proper error checking (verifying the entire string was consumed and the result falls within the valid port range 1–65535) would make failures far more diagnosable.

14.7 HTTP Method Support

Only `GET` is implemented; every other method receives `405 Method Not Allowed` (Chapter 9). Adding `POST` support would introduce the need to parse a request body, which in turn requires applying the same header/body framing logic from Chapter 8 on the *server* side — currently the server never needs to look past the blank line terminating the headers, since GET requests carry no body.

14.8 Summary

None of the above omissions are accidental; each represents a conscious boundary drawn to keep this project's scope focused on the core networking and framing concepts documented in Chapters 2 through 10. Readers looking to extend this project are encouraged to treat this chapter as a roadmap, tackling one item at a time against the existing, tested foundation described in Chapter 13.